

JavaScript

HTML ve CSS’ten sonra sıra sayfayı canlandıran dilde.

YOL HARITASI

Bugünün akışı

Bölüm
Giriş ve yol haritası
Değişkenler ve veri tipleri
Operatörler ve koşullar
Döngüler
Fonksiyonlar ve scope
Diziler ve array metotları

Bölüm
Nesneler
Mola
DOM: seçme ve değiştirme
Event’ler ve form
Asenkron JS ve hata ayıklama
Ödev ve kapanış

ÜNITE

Temeller

Programın yapı taşları: veriyi sakla, karar ver, tekrar et, işi fonksiyona ayır.

Değişkenler | Veri tipleri | Koşullar | Döngüler | Fonksiyonlar

Değişkenler: let ve const

- const: değişmeyecek değer, varsayılan tercih
- let: sonradan değişecek değer
- var: eski yöntem, yeni kodda kullanma
- İsimler camelCase: ogrenciAdi

BENZETME

Değişken, üzerine etiket yapıştırılmış bir kutudur. const kapağı yapıştırılmış kutu, let kapağı açılabilen kutu.

```
const ad = 'Ayşe';
let puan = 80;
puan = puan + 10;
puan += 5; // 95

// ad = 'Ali'; → Hata!
console.log(ad, puan);
// Ayşe 95
```

Veri tipleri

Tip	Örnek	typeof sonucu
string	'merhaba', "dünya"	"string"
number	42, 3.14, -7	"number"
boolean	true, false	"boolean"
undefined	değer verilmemiş	"undefined"
null	bilerek boş	"object" (eski hata)
array	[1, 2, 3]	"object"
object	{ ad: 'Ali' }	"object"

```
const mesaj = `Merhaba ${ad}, ${puan} puan`;
// backtick içinde ${ } ile değişken yerleştirilir
```

02 · OPERATÖRLER

Karşılaştırma ve mantık

- === tipi de kontrol eder, her zaman bunu kullan
- == tip dönüştürür ve sürpriz yapar
- && ve · || veya · ! değil
- Falsy: false, 0, '', null, undefined, NaN. Gerisi truthy

BENZETME

=== kimlik kontrolü: isim ve fotoğraf birlikte bakılır. == sadece "isim benziyor mu" der.

```
console.log(5 === '5');
// false
console.log(5 == '5');
// true (şaşırtıcı!)

const yas = 20;
console.log(yas >= 18 && yas < 65);
// true
console.log(0 || 'boş');
// 'boş'
```

if, else ve üçlü operatör

- if / else if / else: yukarıdan aşağı, ilk doğru olan çalışır
- Üçlü operatör: tek satırlık seçim için
- switch: tek değerin çok durumu, her case sonunda break

BENZETME

Yol ayrımı tabelaları: ilk uyan tabelaya dönersin, kalanlara hiç bakmazsın.

```
const not = 72;
if (not ⩾ 90) {
console.log('Pekiyi');
} else if (not ⩾ 50) {
console.log('Geçti');
} else {
console.log('Kaldı');
}

const d = not ⩾ 50 ? 'Geçti' : 'Kaldı';
```

for, while ve for...of

- for: kaç tur döneceğini biliyorsan
- while: şart doğru olduğu sürece
- for...of: dizinin elemanlarını sırayla gez
- break bitirir, continue o turu atlar

BENZETME

Sınıf yoklaması: listedeki her öğrenci için aynı soruyu sorarsın.

```
for (let i = 1; i ⩽ 3; i++) {
console.log('Tur', i);
}

let n = 3;
while (n > 0) {
console.log(n--);
}

for (const x of ['a', 'b']) {
console.log(x);
}
```

Fonksiyonlar ve arrow function

- Tekrar eden işi bir isme bağla, istediğin zaman çağır
- Parametre girdidir, return çıktıdır
- Arrow function: kısa yazım, callback'lerde çok kullanılır

BENZETME

Fonksiyon bir tarif kartıdır: malzemeyi (parametre) verirsin, yemeği (return) alırsın.

```
function topla(a, b) {
return a + b;
}

const carp = (a, b) ⟹ a * b;
const selam = ad ⟹ `Merhaba ${ad}`;

console.log(topla(2, 3)); // 5
console.log(carp(2, 3)); // 6
console.log(selam('Ali'));
```

Scope: değişken nerede yaşar

- Scope, bir değişkenin görünür olduğu alandır
- let ve const süslü parantez bloğunda yaşar
- Fonksiyonun içi dışarıdan görünmez
- Dış değişkeni içeriden okuyabilirsin

BENZETME

Odalı bir ev: koridor lambası her odadan görünür, odanın içindeki lamba dışarıdan görünmez.

```
const okul = 'Merkez';

function sinif() {
const hoca = 'Ayşe';
console.log(okul); // erişilir
}

sinif();
console.log(hoca);
// ReferenceError!
```

ÜNITE

Dizi ve nesne

Veriyi düzenli tutmak: listeler ve kartlar, ikisi birlikte gerçek veriyi oluşturur.

Diziler | Array metotları | Nesneler

Diziler (array)

- Sıralı liste, indeks 0'dan başlar
- push ve pop: sondan ekle ve çıkar
- length: eleman sayısı
- includes: içinde var mı

> **BENZETME**
> Numaralı dolap sırası: ilk dolabın numarası 0'dır, 1 değil.

```
const meyve = ['elma', 'armut'];

meyve.push('muz');
console.log(meyve[0]);
// 'elma'
console.log(meyve.length);
// 3
console.log(meyve.includes('muz'));
// true
meyve.pop();
```

Array metotları

Metot	Ne yapar	Döndürür
forEach	her elemanda işlem yap	hiçbir şey
map	her elemanı dönüştür	yeni dizi
filter	şarta uyanları seç	yeni dizi
find	ilk uyan elemanı bul	eleman
reduce	hepsini tek değere indir	tek değer
sort	sıralar (dizi değişir)	aynı dizi

```
const s = [1, 2, 3, 4];

s.map(x ⇒ x * 2)
// [2, 4, 6, 8]
s.filter(x ⇒ x > 2)
// [3, 4]
s.reduce((t, x) ⇒ t + x, 0)
// 10
```

Nesneler (object)

- Anahtar: değer çiftleri, nokta ile eriş
- Destructuring: değerleri tek satırda çek
- Spread (...): kopyala ve güncelle
- Gerçek veri çoğunlukla nesne dizisidir

BENZETME

Öğrenci kartı: ad, numara, bölüm gibi alanlar tek kartta durur.

```
const ogr = { ad: 'Ali', yas: 20 };

console.log(ogr.ad);
ogr.yas = 21;

const { ad, yas } = ogr;
const kopya = { ...ogr, sinif: 'A' };

const liste = [ogr, kopya];
liste.forEach(o ⇒ console.log(o.ad));
```

ÜNITE

DOM ve event'ler

Sayfayı canlandırma zamanı: elemanı bul, değiştir, kullanıcıya tepki ver.

DOM seçme | İçerik değiştirme | Event'ler | Form

07 · DOM

DOM: elemanı seçmek

- DOM, tarayıcının HTML'den çıkardığı ağaç yapısı
- querySelector: ilk eşleşen eleman
- querySelectorAll: hepsi (NodeList)
- Seçici CSS ile aynı: #id, .sınıf, etiket

> **BENZETME**
>
> Sayfa bir apartman, selector da kapı numarası: doğru daireyi bulursun.

```
// index.html
// <h1 id="baslik">Merhaba</h1>
// <li class="madde">Elma</li>

const b = document.querySelector('#baslik');
const li = document.querySelectorAll('li');

console.log(b);
console.log(li.length);
```

DOM: içeriği ve görünümü değiştirmek

- textContent: metni değiştir
- classList: add, remove, toggle
- createElement + append: yeni eleman üret
- Stil için style yerine CSS sınıfı kullan

BENZETME

CSS'te hazırladığın kıyafeti (class) elemana giydirir ya da çıkarırsın.

```
const b = document.querySelector('#baslik');
b.textContent = 'Yeni başlık';
b.classList.toggle('kirmizi');

const li = document.createElement('li');
li.textContent = 'Yeni madde';
document.querySelector('ul').append(li);
```

Event'ler: kullanıcıya tepki vermek

- addEventListener(olay, fonksiyon)
- Sık olaylar: click, input, submit, keydown
- Fonksiyon event nesnesini alır: e.target, e.key
- Delegation: ebeveyni dinle, çocuğu e.target ile yakala

BENZETME

Kapı zili: zil çalınca (olay) ne yapılacağını (fonksiyon) önceden söylersin.

```
const btn = document.querySelector('button');
let sayac = 0;

btn.addEventListener('click', () ⇒ {
sayac++;
btn.textContent = `Tıklama: ${sayac}`;
});
```

Form ve input: veriyi almak

- input.value: kutudaki metin
- submit olayında e.preventDefault(): sayfa yenilenmesin
- Boş girişi value.trim() ile kontrol et
- Hepsi birleşince: görev listesi uygulaması

BENZETME

preventDefault: tarayıcının "ben hallederim" refleksini durdurmak.

```
form.addEventListener('submit', (e) ⇒ {
e.preventDefault();
const metin = input.value.trim();
if (!metin) return;

const li = document.createElement('li');
li.textContent = metin;
liste.append(li);
input.value = '';
});
```

ÜNITE

Asenkron JS

Beklerken durma: sunucudan veri al, hataları yakala.

setTimeout | Promise ve fetch | async / await | Hata ayıklama

09 · ASENKRON JS

Asenkron: beklerken durmamak

- JS aynı anda tek iş yapar
- Uzun işler (ağ, zamanlayıcı) arka planda beklenir
- setTimeout: süre dolunca çalıştır
- Sonuç gelince çağrılan fonksiyon: callback

> **BENZETME**
>
> Restoran: garson siparişi mutfağa verir, beklemez, diğer masalara bakar. Yemek hazır olunca haber gelir.

```
console.log('1. Sipariş alındı');

setTimeout(() ⇒ {
console.log('3. Yemek hazır');
}, 2000);

console.log('2. Diğer masaya bakıldı');
// Sıra: 1, 2, 3
```

09 · ASENKRON JS

Promise ve fetch

- fetch bir Promise döner: “sonra cevap vereceğim”
- .then ile zincirle, .catch ile hatayı yakala
- res.json(): cevabı nesneye çevirir
- Test için: jsonplaceholder.typicode.com

BENZETME

Kargo takip numarası: paket henüz yok ama “geldi” ya da “kayboldu” diye haber verecek.

```
const url =
'https://jsonplaceholder.typicode.com/users';

fetch(url)
.then(res ⇒ res.json())
.then(users ⇒ console.log(users[0].name))
.catch(err ⇒ console.log('Hata', err));
```

09 · ASENKRON JS

async / await: aynı iş, daha okunur

- await, Promise bitene kadar o satırda bekler
- await yalnızca async fonksiyon içinde çalışır
- try / catch ile hataları yakala
- res.ok kontrolünü atlama

BENZETME

Aynı tarif, bu sefer adım adım maddeler halinde yazılmış.

```
async function getir() {
try {
const res = await fetch(url);
if (!res.ok) throw new Error('Hata');
const data = await res.json();
console.log(data.length);
} catch (err) {
console.log(err.message);
}
}
getir();
```

Hata mesajını okumak

Hata	Genelde anlamı
ReferenceError	Değişken tanımlı değil ya da scope dışında
TypeError	Yanlış tipte işlem, çoğu zaman eleman null
SyntaxError	Parantez, süslü parantez veya tırnak eksik
Uncaught (in promise)	Async hata yakalanmadı, catch ekle
Hata yok ama sayfa boş	Script yolunu, defer'i ve seçiciyi kontrol et

`console.log()`
Değer ne? Ekrana yaz

`console.table()`
Dizi veya nesneyi tablo gör

`debugger`
Kodu o satırda durdur (F12)

ÖDEV

Görev listesini tamamla

1 · Ekle

Formdan görev ekle, boş girişi engelle

2 · Tamamla

Tıklayınca üstünü çiz (classList.toggle)

3 · Sil

Sil butonu, event delegation ile

4 · Bonus

Görev sayısını göster, API'den örnek görevleri çek

KAPANIŞ

Bugün neler gördük?

- Temeller
- Fonksiyonlar
- Dizi ve nesne
- DOM ve event
- Asenkron JS

Oku. Yaz. Boz. Tekrar dene.